Deadlock Prevention

Deadlock prevention is a strategy used in computer systems to ensure that different processes can run smoothly without getting stuck waiting for each other forever.

Think of it like a traffic system where cars (processes) must move through intersections (resources) without getting into a gridlock.

As we have already discussed that deadlock can only happen if all four of the following conditions are met simultaneously:

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

So we can prevent a Deadlock by eliminating any of the above four conditions. Let's look at how:

1. Eliminate Mutual Exclusion

- Some resources, like a printer, are inherently non-sharable, so this condition is difficult to break.
- However, shareable resources like read-only files can be accessed by multiple processes at the same time.
- For non-sharable resources, prevention through this method is not possible.

2. Eliminate Hold and Wait

Hold and wait is a condition in which a process holds one resource while simultaneously waiting for another resource that is being held by a different process. The process cannot continue until it gets all the required resources.

There are two ways to eliminate hold and wait:

- **By eliminating wait**: The process specifies the resources it requires in advance so that it does not have to wait for allocation after execution starts.
  For Example, Process1 declares in advance that it requires both Resource1 and Resource2.
- **By eliminating hold**: The process has to release all resources it is currently holding before making a new request.
  For Example: Process1 must release Resource2 and Resource3 before requesting Resource1.

3. Eliminate No Pre-emption

No pre-emption means resources can't be taken away once allocated. To prevent this:

- **Processes must release resources voluntarily**: A process gives up resources once it finishes using them.
- **Avoid partial allocation**: If a process requests resources that are unavailable, it must release all currently held resources and wait until all required resources are free.

4. Eliminate Circular Wait

Circular wait happens when processes form a cycle, each waiting for a resource held by the next. To prevent this:

- Impose a strict ordering on resources.
- Assign each resource a unique number.
- Processes can only request resources in increasing order of their numbers.
- This prevents cycles, as no process can go backwards in numbering.

Deadlock Avoidance

BANKER'S ALGORITHM (Very Important)

- Deadlock avoidance is a **dynamic method** in operating systems that ensures the system will **never enter a deadlock state**.
- Instead of letting deadlocks occur and then handling them, the system carefully **analyzes every resource-allocation request in advance** and decides whether to grant or deny it.
- **A request is granted only if it leads the system to a "safe state."**

A **safe state** means there exists at least one sequence in which all processes can complete without leading to deadlock.

An **unsafe state** does not mean deadlock has occurred, but it could possibly occur in the future.

Why is Deadlock Avoidance Needed?

- Deadlocks waste CPU cycles and cause processes to hang indefinitely.
- Unlike **deadlock prevention** (which is too restrictive) or **deadlock detection and recovery** (which allows failure and then fixes it), **avoidance tries to balance efficiency and safety**.
- It allows maximum concurrency **without sacrificing safety**.

Analogy to a Banker

The algorithm is named after a banker who allocates money:

- A **bank knows its total cash reserves** (available resources) and the maximum loan amounts customers (processes) can eventually repay.

- **Before approving a loan**, the **bank checks** if **approving the loan would still leave enough money in reserve** for other customers to potentially receive their loans and repay them in the future. If not, the loan is denied, preventing a bank run (deadlock).

  The Banker's Algorithm is a deadlock avoidance technique in operating systems that prevents deadlocks by ensuring a system is always in a "safe state" before allocating resources.

- **It works by tracking each process's maximum resource needs, current allocations, and available resources to determine if a resource request can be granted without leading the system into a state where processes can't complete.**
- The algorithm **checks if a safe sequence of execution** exists where all processes can eventually finish, thereby preventing deadlocks by denying requests that would lead to an unsafe state.

Key Concepts in Banker's Algorithm

- **Safe State:** There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.
- **Unsafe State:** Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.

Components of the Banker's Algorithm

The following **Data structures** are used to implement the Banker's Algorithm:

Let **'n'** be the number of processes in the system and **'m'** be the number of resource types.

1. Available

- It is a 1-D array of size **'m'** indicating the number of available resources of each type.
- Available[j] = k means there are **'k'** instances of resource type $\mathbf{R_j}$

2. Max

- It is a 2-d array of size '**n*m'** that defines the maximum demand of each process in a system.
- Max[i, j] = k means process $\mathbf{P_i}$ may request at most **'k'** instances of resource type $\mathbf{R_j}$**.**

3. Allocation

- It is a 2-d array of size **'n*m'** that defines the number of resources of each type currently allocated to each process.
- Allocation[i,j]=k means process $\mathbf{P_i}$ is currently allocated **'k'** instances of resource type $\mathbf{R_j}$

4. Need

- It is a 2-d array of size **'n*m'** that indicates the remaining resource need of each process.
- Need [i, j] = k means process $\mathbf{P_i}$ currently needs **'k'** instances of resource type $\mathbf{R_j}$
- Need [i, j] = Max [i, j] – Allocation [i, j]

Allocation specifies the resources currently allocated to process Pi and Need specifies the additional resources that process Pi may still request to complete its task.

Example of Unsafe State

Consider a system with three processes (P1, P2, P3) and 6 instances of a resource. Let's say:

1. The available instances of the resource are: 1
2. The current allocation of resources to processes is:

- P1: 2
- P2: 3
- P3: 1

3. The maximum demand (maximum resources each process may eventually request) is:

- P1: 4
- P2: 5
- P3: 3

In this situation:

Need = Maximum Demand - Allocation

Process	Maximum Demand	Allocation	Need
P1	4	2	2
P2	5	3	2
P3	3	1	2

- P1 may need 2 more resources to complete.
- P2 may need 2 more resource to complete.
- P3 may need 2 more resources to complete.

However, there is only 1 resource available. Even though none of the processes are currently deadlocked, the system is in an unsafe state because there is no sequence of resource allocation that guarantees all processes can complete.

Banker's algorithm consists of a Safety algorithm and a Resource request algorithm.

1. Safety Algorithm
2. Resource Request Algorithm

1. Safety Algorithm

The safety algorithm checks whether the system is in a **safe state**- meaning all processes can complete without causing deadlock.

Steps:

1. Initialize:

- Work = Available (resources currently available)
- Finish[i] = false for all processes (none have finished yet)

2. Look for a process Pi such that:

- Finish[i] = false
- Need[i] ≤ Work (resources required can be satisfied)

3. If such a process is found:

- Pretend to allocate resources to it: Work = Work + Allocation[i]
- Mark the process finished: Finish[i] = true
- Repeat step 2 for remaining processes

4. If all processes are marked finished (Finish[i] = true for all), the system is **safe**.

Essentially, the algorithm simulates process completion to verify that all processes can finish safely.

2. Resource Request Algorithm

This algorithm decides if a process's resource request can be granted safely.

Steps:

1. Check if the request exceeds the process's maximum need: If Request[i] ≤ Need[i], continue; otherwise, **error**.

2. Check if resources are available: If Request[i] ≤ Available, continue; otherwise, the process **waits**.

3. Temporarily allocate the resources:

- Available = Available - Request[i]
- Allocation[i] = Allocation[i] + Request[i]
- Need[i] = Need[i] - Request[i]

4. **Run the Safety Algorithm:**

- If the new state is safe, grant the request.
- If unsafe, roll back and make the process wait.

In short, the resource-request algorithm ensures resources are only granted if the system remains safe, preventing deadlock.

Advantages of avoidance:

- More resource utilization than prevention.
- No need for expensive recovery mechanisms.

Limitations:

- Requires prior knowledge of **maximum resource needs** of each process.

- Extra overhead to check system state on each request.
- May cause **starvation** if processes are repeatedly denied.

Safe state check

Q1. Consider a system with three processes: P0, P1, P2 and three resources R1, R2, R3. Allocation, maximum requirement, and Total instances of all three resources are given below. Is the system in a safe state? If yes, give a safe sequence.

Given:

Allocation

- P0: [0, 1, 0]
- P1: [2, 0, 0]
- P2: [3, 0, 2]

- **Maximum**
  - P0: [7, 5, 3]
  - P1: [3, 2, 2]
  - P2: [9, 0, 2]
- **Total resources**: [10, 5, 7]

Solution:

Process	Allocation	Maximum	Need =Max−Alloc
P0	[0, 1, 0]	[7, 5, 3]	[7, 4, 3]
P1	[2, 0, 0]	[3, 2, 2]	[1, 2, 2]
P2	[3, 0, 2]	[9, 0, 2]	[6, 0, 0]

Total resources: [10, 5, 7]

Compute Available (Total − sum(Alloc))

- Sum allocations = [0+2+3, 1+0+0, 0+0+2] = [5, 1, 2]
- **Available (initial)** = [10,5,7] − [5,1,2] = **[5,4,5]**

Safety-check (Banker's algorithm)

Step	Available (before)	Process chosen (Need ≤ Available)	Allocation added when finished	Available (after)
1	[5,4,5]	P1 (need [1,2,2])	[2,0,0]	[7,4,5]
2	[7,4,5]	P2 (need [6,0,0])	[3,0,2]	[10,4,7]
3	[10,4,7]	P0 (need [7,4,3])	[0,1,0]	[10,5,7]

Result: Safe. **Safe sequence:** P1 → P2 → P0.

Granting/Rejecting a request
(Use the same system as Problem 1)

Q2. Using the system from Problem 1 (same Allocation, Max, Total). Process P1 now requests additional resources: **Request = [1, 0, 2]**. **According to Banker's algorithm, should this request be granted immediately?**

Solution : Available (initial) = [10,5,7] − [5,1,2] = **[5,4,5]**

Process	Allocation	Maximum	Need =Max−Alloc
P0	[0, 1, 0]	[7, 5, 3]	[7, 4, 3]
P1	[2, 0, 0]	[3, 2, 2]	[1, 2, 2]
P2	[3, 0, 2]	[9, 0, 2]	[6, 0, 0]

1. **Check request ≤ Need** for P1:
   - P1 Need = [1,2,2] (from Problem 1).
   - Request [1,0,2] ≤ Need [1,2,2] → **OK**.
2. **Check request ≤ Available** (Available from Problem 1 = [5,4,5]):
   - Request [1,0,2] ≤ Available [5,4,5] → **OK**.
3. **Pretend to allocate** (temporary state):
   - New Allocation(P1) = [2,0,0] + [1,0,2] = **[3,0,2]**.
   - New Need(P1) = [1,2,2] − [1,0,2] = **[0,2,0]**.
   - New Available = [5,4,5] − [1,0,2] = **[4,4,3]**.
4. **Check the safety of this temporary state** with Banker's algorithm:
   - Available = [4,4,3].

- Find a process whose Need ≤ Available:
  - P1 need [0,2,0] ≤ [4,4,3] → P1 can finish → Available becomes [4,4,3] + Allocation(P1) (new) [3,0,2] = **[7,4,5]**.
  - Then P2 need [6,0,0] ≤ [7,4,5] → P2 finishes → Available becomes [7,4,5] + [3,0,2] = [10,4,7].
  - Then P0 need [7,4,3] ≤ [10,4,7] → P0 finishes.
- All processes can finish → the state is **safe**.

Answer: Yes — the request **can be granted** (after the temporary check, the system remains in a safe state).

Q3. Single-resource

Suppose there is **one resource type** (R) with total 12 instances. Three processes have:

- Allocation: P0 = 3, P1 = 2, P2 = 6
- Maximum: P0 = 7, P1 = 4, P2 = 9

Questions:

1. Is the system safe right now?
2. If P1 requests 1 additional instance (so request = 1), should it be granted?

Given

Process	Allocation (R)	Maximum (R)	Need (Max−Alloc)
P0	3	7	4
P1	2	4	2

P2	6	9	3

Total (R) = 12

Compute Available

- Sum allocations = 3 + 2 + 6 = 11
- **Available (initial)** = 12 − 11 = **1**

Safety-check

Step	Available (before)	Any process with Need ≤ Available?	Reason
1	1	None	P0 need 4; P1 need 2; P2 need 3 → all > 1

Result: Not safe (no process can finish now).

(ii) If P1 requests 1

Check	Value	Result
Request ≤ Need?	1 ≤ 2 → OK	
Request ≤ Available?	1 ≤ 1 → OK	

Pretend allocate

- New Allocation(P1) = 2+1 = 3
- New Need(P1) = 2−1 = 1
- New Available = 1−1 = 0

Now, no process has Need ≤ 0 → **unsafe**, so **deny** the request.

PRACTICE QUESTION

Q1. Consider a system with five processes: P0, P1, P2, P3, P4 and three resources R1, R2, and R3. Allocation, maximum requirement, and Total instances of all three resources are given below. Is the system in a safe state? If yes, give a safe sequence. **Total resources**: [10, 5, 7]

Allocation

- P0: [0,1,0]
- P1: [2,0,0]
- P2: [3,0,2]
- P3: [2,1,1]
- P4: [0,0,2]

Maximum

- P0: [7,5,3]
- P1: [3,2,2]
- P2: [9,0,2]
- P3: [2,2,2]
- P4: [4,3,3]

Q2. Using the Problem 3 system, process P4 makes a request: **Request = [3, 3, 0]**. Should the request be granted?

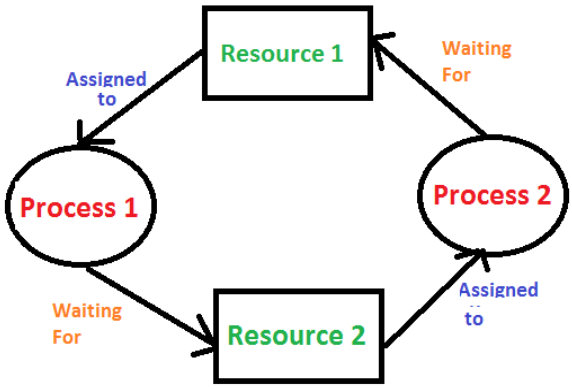
DEADLOCK DETECTION

- Deadlock detection in an operating system (OS) is the process of identifying when a deadlock has occurred, which is a state where multiple processes are stuck waiting for each other to release resources.
- For systems with multiple instances of each resource type, a variation of the Banker's algorithm can be used to determine if the current state is safe or unsafe, implying a potential deadlock.

Methods for Deadlock Detection

1. If Resources Have a Single Instance

- In this case for Deadlock detection, we can run an algorithm to check for the cycle in the **Resource Allocation Graph.**
- The presence of a cycle in the graph is a sufficient condition for deadlock.



In the above diagram, resource 1 and resource 2 have single instances. There is a cycle R1 → P1 → R2 → P2. So, **Deadlock is Confirmed.**

2. If There are Multiple Instances of Resources

- Detection of the cycle is necessary but not a sufficient condition for deadlock detection, in this case, the system may or may not be in deadlock varies according to different situations.

- For systems with multiple instances of resources, algorithms like ***Banker's Algorithm*** can be adapted to periodically check for deadlocks.

Banker's Algorithm variation:

- ***When to use:*** This approach is used for systems with multiple instances of each resource type.
- ***How it works:*** The algorithm simulates resource allocation to see if a "safe state" can be reached. It checks if there is a sequence of process executions that would allow all processes to finish without a deadlock. If no such safe sequence exists, it indicates an unsafe state and a deadlock has occurred.

3. Wait-For Graph Algorithm

- The ***Wait-For Graph Algorithm*** is a deadlock detection algorithm used to detect deadlocks in a system where resources can have multiple instances.
- The algorithm works by constructing a Wait-For Graph, which is a ***directed graph that represents the dependencies between processes and resources.***

Wait For Graph Deadlock Detection

A Deadlock is a situation where a set of processes are blocked as each process in a Distributed system is holding some resources and that acquired resources are needed by some other processes.

Wait-for-Graph Algorithm

- It is a variant of the Resource Allocation graph.
- **In this algorithm, we only have processes as vertices in the graph. If the Wait-for-Graph contains a cycle then we can say the system is in a Deadlock state.**
- We need to remove resources while converting from Resource Allocation Graph to Wait-for-Graph.

Resource Allocation Graph: Contains Processes and Resources.

Wait-for-Graph: Contains only Processes after removing the Resources while conversion from Resource Allocation Graph.

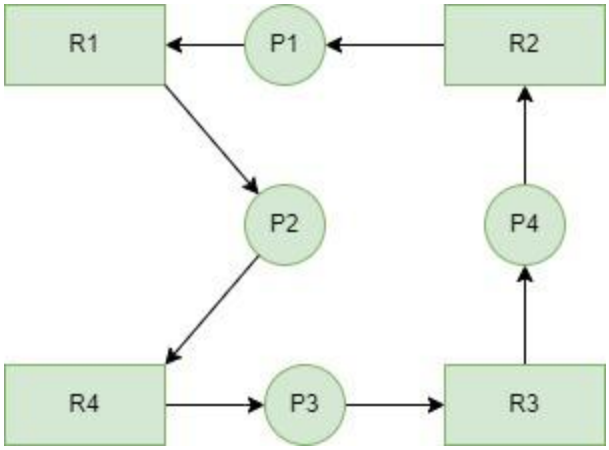
Algorithm

Below are the steps to follow:

- **Step 1:** Take the first process (Pi) from the resource allocation graph and check the path in which it is acquiring resource (Ri), and start a wait-for-graph with that particular process.
- **Step 2:** Make a path for the Wait-for-Graph in which there will be no Resource included from the current process (Pi) to next process (Pj), from that next process (Pj) find a resource (Rj) that will be acquired by next Process (Pk) which is released from Process (Pj).
- **Step 3:** Repeat Step 2 for all the processes.
- **Step 4:** After completion of all processes, if we find a closed-loop cycle then the system is in a deadlock state, and deadlock is detected.

Example 1:

Consider a Resource Allocation Graph with 4 Processes P1, P2, P3, P4, and 4 Resources R1, R2, R3, R4. Find if there is a deadlock in the Graph using the Wait for Graph-based deadlock detection algorithm.

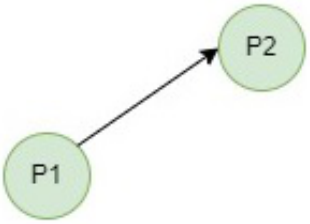


Step 1

First take Process P1 which is waiting for Resource R1, resource R1 is acquired by Process P2, Start a Wait-for-Graph for the above Resource Allocation Graph.

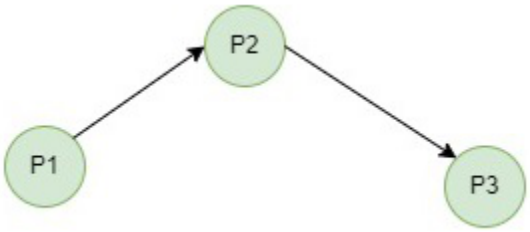
Step 2

Now we can observe that there is a path from P1 to P2 as P1 is waiting for R1 which is been acquired by P2. Now the Graph would be after removing resource R1 looks like.



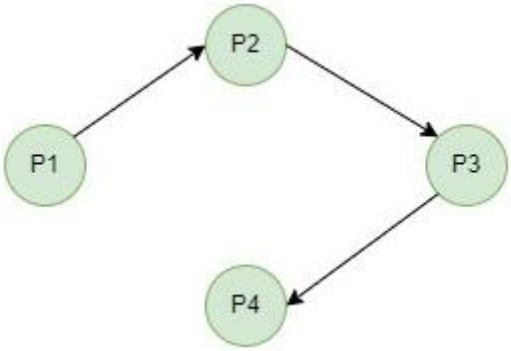
Step 3

From P2 we can observe a path from P2 to P3 as P2 is waiting for R4 which is acquired by P3. So make a path from P2 to P3 after removing resource R4 looks like.



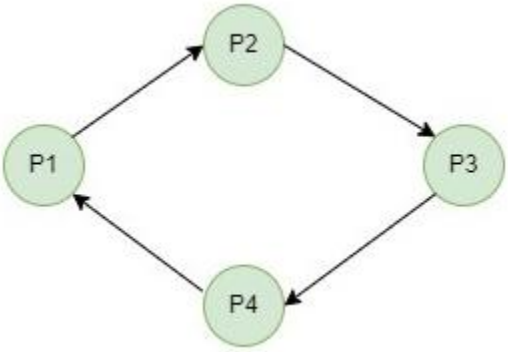
Step 4

From P3 we find a path to P4 as it is waiting for P3 which is acquired by P4. After removing R3 the graph looks like this.



Step 5

Here we can find Process P4 is waiting for R2 which is acquired by P1. So finally the Wait-for-Graph is as follows:



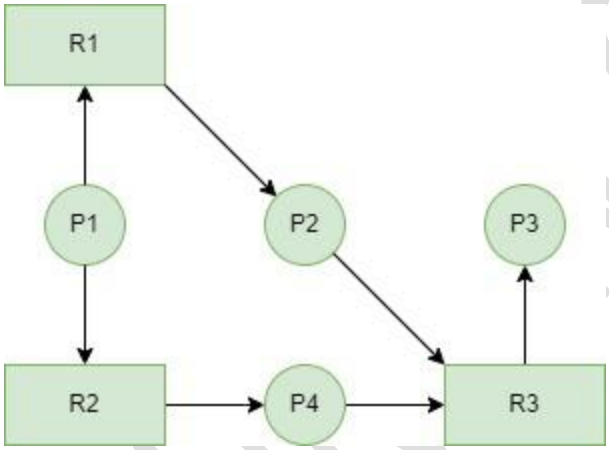
Step 6

Finally In this Graph, we found a cycle as the Process P4 again came back to the Process P1 which is the starting point (i.e., it's a closed-loop).

So, According to the Algorithm if we found a closed loop, then the system is in a deadlock state. So here we can say the system is in a deadlock state.

Example 2:

Now consider another Resource Allocation Graph with 4 Processes P1, P2, P3, P4, and 3 Resources R1, R2, R3. Find if there is a deadlock in the Graph using the Wait for Graph-based deadlock detection algorithm.

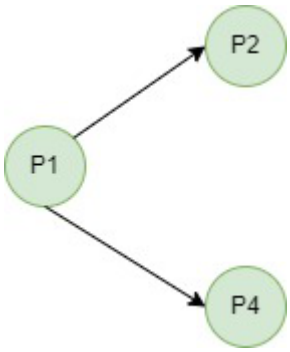


Step 1

First take Process P1 which is waiting for Resource R1, resource R1 is acquired by Process P2, Start a Wait-for-Graph for the above Resource Allocation Graph.

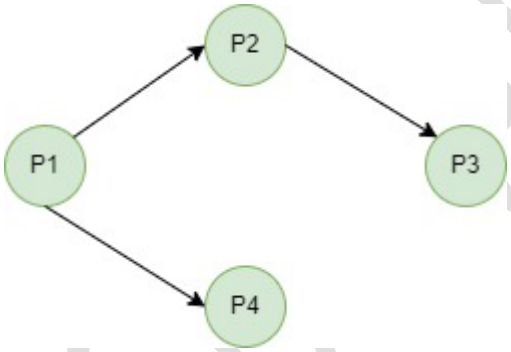
Step 2

Now we can observe that there is a path from P1 to P2 and also from P1 to P4 as P1 is waiting for R1 which is been acquired by P2 and P1 is also waiting for R2 which is acquired by P4. Now the Graph would be after removing resources R1 and R2 looks like.



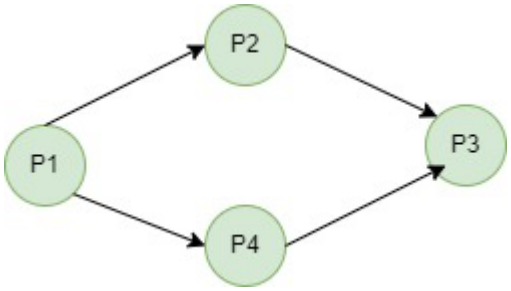
Step 3

From P2 we can observe a path from P2 to P3 as P2 is waiting for R3 which is acquired by P3. So make a path from P2 to P3 after removing resource R3 looks like.



Step 4

Here we can find Process P4 is waiting for R3 which is acquired by P3. So finally the Wait-for-Graph looks like after removing Resource R3 looks like.



Step 5

In this Graph, we don't find a cycle as no process came back to the starting point (i.e., there is no closed loop). So, According to the Algorithm if we found a closed loop, then the system is in a deadlock state. But here we didn't find any closed loop so the system is not in a deadlock state. The system is in a safe state.

In Example 2, even though it looks like a loop but there is no process that has reached the first process, or starting point again. So there is no closed loop.

Deadlock Recovery

A traditional operating system such as Windows doesn't deal with deadlock recovery as it is a time and space-consuming process. Real-time operating systems use Deadlock recovery.

- **Killing The Process:** Killing all the processes involved in the deadlock. Killing process one by one. After killing each process check for deadlock again and keep repeating the process till the system recovers from deadlock. Killing all the processes one by one helps a system to break circular wait conditions.
- **Process Rollback**: Rollback deadlocked processes to a previously saved state where the deadlock condition did not exist. It requires checkpointing to periodically save the state of processes.

- **Resource Preemption:** Resources are preempted from the processes involved in the deadlock, and preempted resources are allocated to other processes so that there is a possibility of recovering the system from the deadlock. In this case, the system goes into starvation.
- **Concurrency Control:** *Concurrency control mechanisms* are used to prevent data inconsistencies in systems with multiple concurrent processes.
- These mechanisms ensure that concurrent processes do not access the same data at the same time, which can lead to inconsistencies and errors.
- Deadlocks can occur in concurrent systems when two or more processes are blocked, waiting for each other to release the resources they need. This can result in a system-wide stall, where no process can make progress. Concurrency control mechanisms can help prevent deadlocks by managing access to shared resources and ensuring that concurrent processes do not interfere with each other.

Advantages of Deadlock Detection and Recovery

- **Improved System Stability:** Deadlocks can cause system-wide stalls, and detecting and resolving deadlocks can help to improve the stability of the system.
- **Better Resource Utilization:** By detecting and resolving deadlocks, the operating system can ensure that resources are efficiently utilized and that the system remains responsive to user requests.
- **Better System Design**: Deadlock detection and recovery algorithms can provide insight into the behavior of the system and the relationships between processes and resources, helping to inform and improve the design of the system.

Disadvantages of Deadlock Detection and Recovery

- **Performance Overhead:** Deadlock detection and recovery algorithms can introduce a significant overhead in terms of performance, as the system must regularly check for deadlocks and take appropriate action to resolve them.
- **Complexity:** Deadlock detection and recovery algorithms can be complex to implement, especially if they use advanced techniques such as the Resource Allocation Graph or Timestamping.
- **False Positives and Negatives:** Deadlock detection algorithms are not perfect and may produce false positives or negatives, indicating the presence of deadlocks when they do not exist or failing to detect deadlocks that do exist.
- **Risk of Data Loss**: In some cases, recovery algorithms may require rolling back the state of one or more processes, leading to data loss or corruption.